

Technical Documentation

RAG Customer Support Assistant

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) combines a search system with an LLM. Instead of relying on the LLM's training data alone, we first search a knowledge base for relevant text, then give that text to the LLM as context. This grounds the answer in real documents and prevents hallucination.

Why do we need it?

- LLMs cannot access private company documents
- RAG lets us use our own PDF without retraining
- Cheaper and faster than fine-tuning a model

Use Case

A company uploads its FAQ/policy PDF. Customers ask questions in natural language. The bot finds the most relevant policy text and answers from it. Questions the bot cannot answer are passed to a human support agent.

2. How the System Works

Phase 1 — Ingestion (runs once)

The PDF is loaded page by page. Each page is split into 500-character chunks with a 50-character overlap so sentences are not cut in half. Each chunk is converted into a 384-dimensional vector using the all-MiniLM-L6-v2 model and saved to ChromaDB on disk.

Phase 2 — Query handling (runs every time a user asks)

The user's question is embedded with the same model. ChromaDB finds the 3 most similar chunks. A LangGraph workflow then decides: if chunks were found, the LLM reads them and writes an answer. If no chunks were found or the user asked for a human, the workflow pauses and a human agent types the answer instead.

3. Design Decisions

Chunk size: 500 characters

Small enough to be precise in retrieval, large enough to preserve sentence meaning. The 50-character overlap prevents important context being cut at a boundary.

top-k = 3 chunks

Enough context for most FAQ answers. Keeping it at 3 avoids hitting LLM token limits and keeps costs low.

all-MiniLM-L6-v2 embeddings

Free and runs locally with no API key. Fast and accurate enough for English FAQ text.

ChromaDB

No server setup required. Data is persisted to a local folder. Simple Python API.

Prompt design

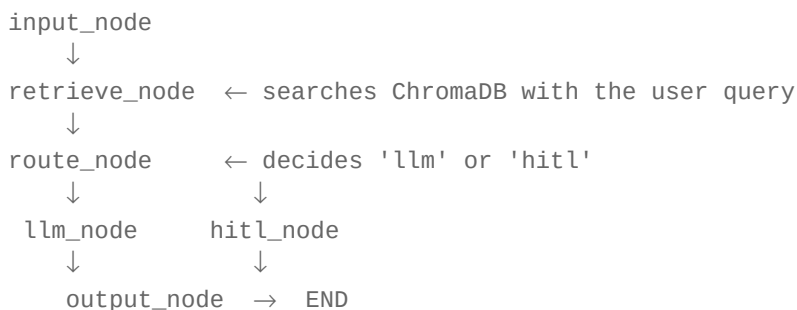
The system prompt says 'Answer using ONLY the context below.' This prevents the LLM from inventing information not in the PDF.

LangGraph over a simple chain

An explicit graph makes the conditional routing (LLM vs HITL) clean and easy to read. Adding new routes later requires just one new node and one new edge.

4. LangGraph Workflow

The workflow is a StateGraph with 6 nodes. All nodes share one state dictionary and each returns only the fields it changes.



The conditional edge after `route_node` reads `state['route']` and directs execution to either `llm_node` or `hitl_node`. Both paths then merge at `output_node`.

5. Conditional Routing

Two simple rules decide whether to escalate:

- Rule 1: if ChromaDB returns zero chunks → escalate (the bot has nothing to say)

- Rule 2: if the query contains the word 'human' → escalate immediately

In a production system this could be extended with a confidence threshold (e.g. escalate if the best similarity distance score is below 0.4).

6. HITL Implementation

HITL (Human-in-the-Loop) means a real person steps in to answer when the system cannot. In this project, the `hitl_node` pauses the graph and uses Python's `input()` to let a human type a response in the terminal. That response is stored in `state['answer']` and returned to the user in the same format as an LLM answer.

Benefits

- Handles questions the PDF does not cover
- Prevents the LLM from guessing or hallucinating on unknown topics
- Easy to upgrade to a real ticketing system (Zendesk, Slack) later

Limitations

- Requires a human at the terminal — not suitable for 24/7 unattended use
- No logging of escalated queries out of the box (easy to add)

7. Challenges and Trade-offs

Retrieval accuracy vs speed

More chunks (higher top-k) gives the LLM more context but increases prompt size and API cost. top-k=3 is a practical balance.

Chunk size vs context quality

Larger chunks reduce retrieval precision; smaller chunks may cut sentences. 500 chars with 50 overlap works well for FAQ-style text.

Local model vs API embeddings

all-MiniLM-L6-v2 is free and private. OpenAI ada-002 is slightly more accurate but costs money and sends data to an external API.

Cost vs performance

GPT-3.5-turbo is cheap and fast. GPT-4 handles complex reasoning better but costs ~20x more.

8. Testing Strategy

Unit tests (one file per module)

- test_chunker.py — check chunk count > 0 and each chunk length <= 550
- test_retriever.py — mock ChromaDB; confirm top-3 returned
- test_route_node.py — empty chunks → 'hitl'; non-empty → 'llm'

Sample queries to test manually

"What is your refund policy?"

Expected: Should retrieve policy chunks and return an LLM answer.

"How do I reset my password?"

Expected: Should retrieve account chunks and return an LLM answer.

"xyzyz abc nonsense"

Expected: No chunks found → HITL escalation triggered.

"I want to speak to a human agent"

Expected: 'human' in query → HITL escalation triggered.

9. Future Enhancements

Multi-document support

Tag each chunk with its source document. Filter ChromaDB search by tag.

Feedback loop

Ask the user to rate each answer. Store poor ratings to improve the knowledge base.

Conversation memory

Include the last 2–3 Q&A; turns in the prompt for follow-up question support.

Web UI

Replace the CLI with a FastAPI backend and a simple HTML/JS frontend.

Async HITL

Replace input() with a Slack or Zendesk ticket. Poll for agent response asynchronously.

Confidence threshold

Escalate if the best ChromaDB similarity distance score is below a set threshold.